

LẬP TRÌNH WEB

Chương 3

JavaScript cơ bản

GV: Bùi Văn Thương

Email: thuong.bui@ut.edu.vn

CÔNG CỤ HỌC TẬP

Code editor:
Microsoft Visual
Studio Code



Extension: Live
Server



JavaScript

NỘI DUNG CHÍNH



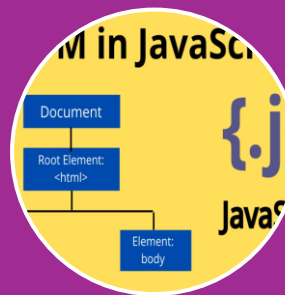
Giới thiệu về
JavaScript



Cách sử
dụng
JavaScript
trong HTML



Cú pháp cơ
bản của
JavaScript



Thao tác với
HTML DOM



AJAX



GIỚI THIỆU CƠ BẢN VỀ JAVASCRIPT

JavaScript là ngôn ngữ lập trình phổ biến nhất thế giới dành cho trình duyệt.

- **Ngôn ngữ thông dịch:** Chạy trực tiếp trên trình duyệt mà không cần biên dịch phức tạp.
- **Hướng sự kiện** (Event-driven): Trang web phản ứng khi ta click chuột, gõ phím hoặc cuộn trang.
- **Đa năng:** Ngày nay JS không chỉ làm Front-end mà còn làm được cả Back-end (Node.js), App di động và AI.

GIỚI THIỆU CƠ BẢN VỀ JAVASCRIPT

Lịch sử của JavaScript

1995

Brendan Eich nhận nhiệm vụ là tạo ra một ngôn ngữ lập trình kịch bản nhẹ, có thể chạy trực tiếp trên trình duyệt để làm cho các trang web trở nên “sống động” hơn. Ban đầu, nó có tên là Mocha, sau đó đổi thành LiveScript và cuối cùng là cái tên JavaScript

1997

JavaScript chính là một “hiện thực hóa” (implementation) nổi tiếng nhất của tiêu chuẩn ECMAScript. Điều này đảm bảo rằng một đoạn mã JavaScript viết theo chuẩn ES sẽ có thể chạy nhất quán trên các trình duyệt khác nhau.

2016

Phiên bản ES6 (ECMAScript 2015): Được coi là cuộc cách mạng lớn nhất, ES6 giới thiệu hàng loạt cú pháp và tính năng mới giúp việc lập trình trở nên dễ dàng và mạnh mẽ hơn rất nhiều.

GIỚI THIỆU CƠ BẢN VỀ JAVASCRIPT

Các ứng dụng phổ biến của JavaScript trên web hiện nay

Tạo website tương tác người dùng:

- Form validation (Kiểm tra biểu mẫu)
- Hiệu ứng UX/UI
- AJAX

Phát triển ứng dụng web front-end và back-end

- Frameworks Front-end nổi bật: React, Angular, Vue.js, NextJS, ...
- Phát triển Back-end với Node.js: ExpressJS, NestJS, ...

CÁCH SỬ DỤNG JAVASCRIPT TRONG HTML

Có 3 cách chính để đưa JS vào trang HTML

Cách 1 - Inline (Trực tiếp trong thẻ): Dùng cho các hiệu ứng cực ngắn.

```
<button onclick="alert('Hello!')">Bấm vào đây</button>
```

Cách 2 - Internal (Trong thẻ <script>): Thường đặt ở cuối thẻ <body> để không làm chậm việc tải giao diện.

```
<script>  
    console.log("Chào mừng bạn đến với JS!");  
</script>
```

Cách 3 - External (File bên ngoài): Giúp code sạch sẽ và dễ quản lý.

```
<head>  
    <!-- Các thẻ khác -->  
    <script src="script.js"></script>  
</head>
```

Khai báo biến

- let: Biến có thể thay đổi giá trị.
- const: Hằng số (không thể thay đổi).
- Lưu ý: Không nên dùng var vì nó đã lỗi thời và dễ gây lỗi logic. Luôn ưu tiên dùng const, chỉ dùng let khi thực sự cần thay đổi.

Kiểu dữ liệu (Data Types)

- String: Chữ cái (đặt trong " " hoặc ' ').
- Number: Số (không có ngoặc).
- Boolean: Chỉ có true (đúng) hoặc false (sai).
- Object & Array: Các cấu trúc dữ liệu phức tạp hơn.

Toán tử cơ bản (Operators)

Toán tử	Ý nghĩa
+, -, *, /	Phép cộng, trừ, nhân, chia
**	Lũy thừa
%	Chia lấy dư
++, --	Phép tăng +1 và giảm -1
=, +=, -=, *=, /=, % =, ** =	Các phép gán
==, >, <, >=, <=, !=	Các toán tử so sánh
===	So sánh bằng cả giá trị và kiểu dữ liệu
!==	So sánh khác cả về giá trị và khác kiểu dữ liệu
&&, , !	Các phép logic gồm: và, hoặc, phủ định

Một số toán tử đặc biệt:

- ?? (Nullish Coalescing): Trả về giá trị bên trái nếu khác null/undefined, ngược lại trả về bên phải.

```
let result = a ?? b;  
// Nếu a khác null và undefined → result = a  
// Nếu a là null hoặc undefined → result = b
```

- ?. (Optional Chaining): Truy cập thuộc tính an toàn, tránh lỗi khi đối tượng null/undefined.

```
let user = {};  
console.log(user.info?.name); // undefined
```

- Toán tử ba ngôi ?: Viết gọn cho if...else

```
let age = 20;  
let status = (age >= 18) ? "Người lớn" : "Trẻ em";  
console.log(status); // "Người lớn"
```

Cấu trúc điều khiển (Logic)

JavaScript có các cấu trúc điều khiển cơ bản hoạt động giống với hầu hết các ngôn ngữ lập trình khác:

- Các cấu trúc rẽ nhánh if...else, switch...case.
- Vòng lặp for, while, do while.
- Các lệnh nhảy break, continue.

Sử dụng hàm trong JavaScript

```
// Cách truyền thống
function tinhTong(a, b) {
    return a + b;
}

// Cách hiện đại (Arrow Function)
const tinhTong2 = (a, b) => a + b;
const chao = () => {
    console.log("Hello from Arrow Function!");
};

// Gọi hàm
chao();
console.log(tinhTong(5, 10));
console.log(tinhTong2(10, 20));
```

Mở trình duyệt và bấm F12 ----
> chọn phần Console để xem
kết quả chạy code JavaScript



```
Hello from Arrow Function!
15
30
```

Một số hàm xử lý số trong JavaScript

Hàm / Phương thức	Mô tả
Number()	Chuyển giá trị sang kiểu số.
parseInt()	Chuyển chuỗi sang số nguyên.
parseFloat()	Chuyển chuỗi sang số thực.
isNaN()	Kiểm tra giá trị có phải NaN không.
isFinite()	Kiểm tra giá trị có hữu hạn không.
Math.round()	Làm tròn số.
Math.floor()	Làm tròn xuống.
Math.ceil()	Làm tròn lên.
Math.random()	Sinh số ngẫu nhiên từ 0 đến < 1 .
Math.max() / Math.min()	Lấy giá trị lớn nhất / nhỏ nhất.

Hàm xử lý chuỗi trong JavaScript

Hàm / Phương thức	Mô tả
String()	Chuyển giá trị sang chuỗi.
toUpperCase() / toLowerCase()	Chuyển chữ hoa / thường.
trim()	Xóa khoảng trắng đầu và cuối chuỗi.
slice(start, end)	Cắt chuỗi theo vị trí.
substring(start, end)	Cắt chuỗi (không nhận số âm).
replace() / replaceAll()	Thay thế ký tự/chuỗi.
split()	Tách chuỗi thành mảng.
includes()	Kiểm tra chuỗi con.
startsWith() / endsWith()	Kiểm tra bắt đầu / kết thúc.

Hàm xử lý mảng trong JavaScript

Hàm / Phương thức	Mô tả
push() / pop()	Thêm / xóa phần tử cuối mảng.
shift() / unshift()	Xóa / thêm phần tử đầu mảng.
concat()	Nối mảng.
slice()	Cắt mảng.
splice()	Thêm / xóa phần tử bất kỳ.
indexOf() / lastIndexOf()	Tìm vị trí phần tử.
includes()	Kiểm tra phần tử tồn tại.

Hàm / Phương thức	Mô tả
forEach()	Lặp qua mảng.
map()	Tạo mảng mới từ mảng cũ.
filter()	Lọc phần tử theo điều kiện.
reduce()	Tính toán giá trị tích lũy.
sort()	Sắp xếp mảng.
find() / findIndex()	Tìm phần tử / vị trí phần tử.

Hàm xử lý đối tượng (Object) trong JavaScript

Hàm / Phương thức	Mô tả
Object.keys()	Lấy danh sách key.
Object.values()	Lấy danh sách value.
Object.entries()	Lấy mảng [key, value].
Object.assign()	Sao chép thuộc tính từ object khác.
Object.freeze()	Đóng băng object (không cho sửa).
Object.seal()	Khóa thêm/xóa thuộc tính.

Hàm thời gian trong JavaScript

Hàm / Phương thức	Mô tả
Date()	Lấy thời gian hiện tại.
getFullYear()	Lấy năm.
getMonth()	Lấy tháng (0–11).
getDate()	Lấy ngày.
getDay()	Lấy thứ (0–6).
getHours() / getMinutes() / getSeconds()	Lấy giờ / phút / giây.
setFullYear() / setMonth() / setDate()	Thiết lập thời gian.

Hàm tiện ích khác trong JavaScript

Hàm / Phương thức	Mô tả
setTimeout()	Thực thi sau một khoảng thời gian.
setInterval()	Lặp lại sau mỗi khoảng thời gian.
clearTimeout() / clearInterval()	Hủy hẹn giờ.
alert() / confirm() / prompt()	Hộp thoại trình duyệt.
JSON.parse() / JSON.stringify()	Chuyển đổi giữa JSON và Object.

Đồng bộ và bất đồng bộ trong JavaScript

Đồng bộ là gì?

Mặc định, JavaScript là ngôn ngữ đơn luồng (single-threaded). Nghĩa là nó chạy từng dòng code một, từ trên xuống dưới. Dòng 1 phải xong thì dòng 2 mới bắt đầu. Nếu dòng 1 là một việc tốn thời gian (ví dụ: tải một tấm ảnh 50MB), toàn bộ trang web sẽ bị "đứng hình" (freeze) cho đến khi ảnh tải xong.

Bất đồng bộ là gì?

Bất đồng bộ cho phép JavaScript bắt đầu một việc tốn thời gian, sau đó nhảy sang làm việc khác ngay lập tức mà không cần đợi việc kia xong. Khi việc tốn thời gian đó hoàn tất, nó sẽ thông báo lại cho chúng ta.

Các công cụ xử lý Bất đồng bộ

Thế hệ 1: Callback (Hàm gọi lại)

```
function layDuLieu(callback) {  
    setTimeout(() => { // Giả lập chờ 2 giây  
        console.log("1. Đã lấy dữ liệu");  
        callback();  
    }, 2000);  
}  
layDuLieu(() => {  
    console.log("2. Hiển thị dữ liệu");  
});  
console.log("3. Chạy trước layDuLieu");
```

3. Chạy trước layDuLieu

1. Đã lấy dữ liệu

2. Hiển thị dữ liệu

Vấn đề: Nếu có nhiều việc lồng nhau, ta sẽ rơi vào **Callback Hell** (Địa ngục callback), code sẽ bị đẩy sâu vào bên phải như một cái kim tự tháp, cực kỳ khó đọc.

Các công cụ xử lý Bất đồng bộ

Thế hệ 2: Promise (Lời hứa)

Ra đời trong phiên bản ES6 để giải quyết Callback Hell. Một Promise sẽ có 3 trạng thái:

- Pending: Đang chờ (Lời hứa đang thực hiện).
- Fulfilled: Thành công (Thực hiện lời hứa).
- Rejected: Thất bại (Thất hứa/Có lỗi).

```
const loiHua = new Promise((resolve, reject) => {  
  let thanhCong = true;  
  if (thanhCong) resolve("Lấy dữ liệu thành công!");  
  else reject("Lỗi rồi!");  
});  
  
loiHua  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

Ví dụ về Promise:

```
const login = (username, password) => {  
  // Trả về một "Lời hứa" (Promise)  
  return new Promise((resolve, reject) => {  
    console.log("Đang kiểm tra thông tin... vui lòng đợi.");  
    setTimeout(() => {  
      if (username === "admin" && password === "123456") {  
        // Nếu đúng, thực hiện lời hứa (resolve)  
        resolve({  
          status: 200,  
          user: "Nguyễn Văn Admin",  
          message: "Đăng nhập thành công!"  
        });  
      } else {  
        // Nếu sai, thất hứa (reject)  
        reject("Tên đăng nhập hoặc mật khẩu không đúng!");  
      }  
    }, 2000); // Giả lập server xử lý trong 2 giây  
  });  
};
```

Ví dụ về Promise:

```
// --- CÁCH SỬ DỤNG ---
login("admin", "123456")
  .then((data) => {
    // Chạy vào đây nếu resolve() được gọi
    console.log("Chúc mừng:", data.user);
    console.log("Thông báo:", data.message);
  })
  .catch((error) => {
    // Chạy vào đây nếu reject() được gọi
    console.error("Lỗi:", error);
  })
  .finally(() => {
    // Luôn luôn chạy dù thắng hay thua
    console.log("Kết thúc quy trình đăng nhập.");
  });
```

Đang kiểm tra thông tin... vui lòng đợi.

Chúc mừng: Nguyễn Văn Admin

Thông báo: Đăng nhập thành công!

Kết thúc quy trình đăng nhập.

Các công cụ xử lý Bất đồng bộ

Thế hệ 3: Async / Await (Chuẩn Pro)

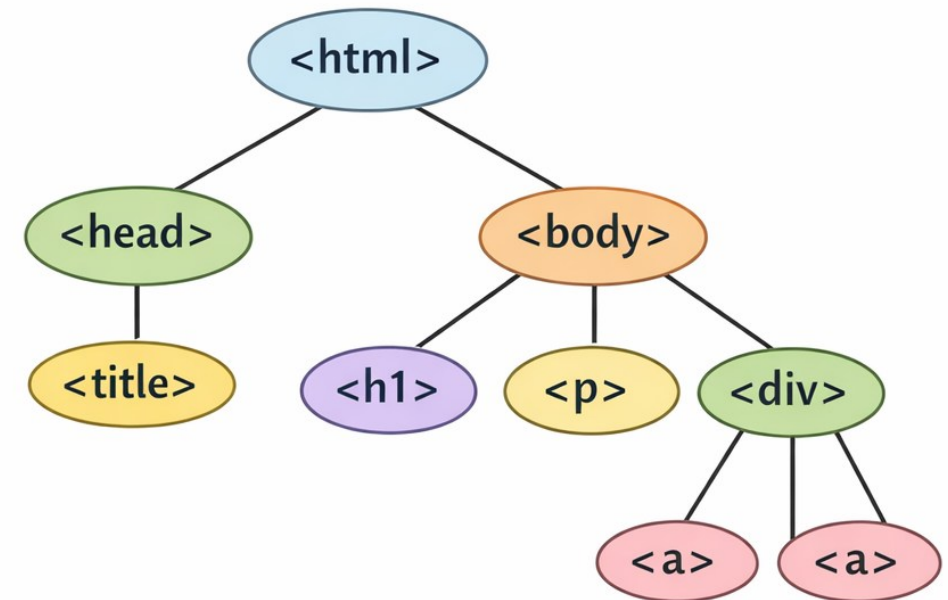
Đây là cách hiện đại nhất. Nó thực chất là "vỏ bọc" cho Promise nhưng giúp code của chúng ta trông như code đồng bộ.

```
async function thucHienCongViec() {  
  try {  
    console.log("Bắt đầu...");  
    // Dừng lại ở đây đợi Promise xong mới chạy tiếp dòng dưới  
    const data = await fetch('https://api.example.com/data');  
    const json = await data.json();  
    console.log("Xong rồi nè:", json);  
  } catch (err) {  
    console.log("Có lỗi:", err);  
  }  
}  
thucHienCongViec();
```

HTML DOM (Document Object Model) là một chuẩn cho phép truy xuất và thao tác trên các thành phần HTML bằng mã lệnh JavaScript như thay đổi nội dung, cấu trúc, cũng như kiểu dáng của trang web một cách linh hoạt.

- Document: Chính là file HTML.
- Object: Mỗi thẻ (<div>, <h1>, <a>) đều biến thành một "đối tượng" mà JS có thể cầm, nắm, thay đổi.
- Model: Cách sắp xếp phân cấp giữa các thẻ (cha, con, anh em).

DOM giúp các nhà phát triển tạo ra các trang web động, thay đổi nội dung và thuộc tính của các phần tử HTML mà không cần tải lại trang.



Các phương thức chọn phần tử HTML

Phương thức	Mô tả
<code>document.getElementById(id)</code>	Tìm theo id
<code>document.getElementsByTagName(name)</code>	Tìm theo tên thẻ
<code>document.getElementsByClassName(name)</code>	Tìm theo tên class
<code>document.querySelector(selector)</code>	Tìm phần tử đầu tiên khớp selector
<code>document.querySelectorAll(selector)</code>	Tìm tất cả phần tử khớp selector

Thay đổi nội dung phần tử HTML:

- `.innerHTML`: Thay đổi cả nội dung và thẻ HTML bên trong.
- `.innerText` hoặc `.textContent`: Chỉ thay đổi phần chữ (Text) thuần túy.

Thay đổi thuộc tính (Attributes):

- `element.src = "anh-moi.jpg";`
- `element.href = "https://google.com";`
- `element.setAttribute('data-id', '123');`

THAO TÁC VỚI HTML DOM

Ví dụ

```
<body>
  <p id="demo">Hello</p>
  <p class="text">World</p>
  <input type="text" name="username" value="John">

  <script>
    console.log(document.getElementById("demo").innerText);
    console.log(document.getElementsByTagName("p")[1].innerText);
    console.log(document.getElementsByClassName("text")[0].innerText);
    console.log(document.querySelector(".text").innerText);
    console.log(document.getElementsByName("username")[0].value);
  </script>
</body>
```

Kết quả

Hello
World
World
World
John

Thao tác với CSS:

- Cách trực tiếp (Inline style):
 - `element.style.backgroundColor = "blue";` (Lưu ý: Dùng camelCase thay cho dấu gạch ngang).
- Cách chuyên nghiệp (classList):
 - `element.classList.add('active');` (Thêm một class đã định nghĩa sẵn trong CSS).
 - `element.classList.toggle('dark-mode');` (Bật/tắt class - cực kỳ hữu ích làm Dark Mode).

Event:

Lắng nghe Sự kiện với phương thức `addEventListener()`. Các sự kiện phổ biến: `click`, `mouseover` (di chuột vào), `keyup` (gõ phím), `submit` (gửi form).

Ví dụ: tạo chế độ Light/Dark Mode

Mã HTML

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Chế độ Sáng / Tối</h1>
    <p>Đây là ứng dụng demo đơn giản dùng JavaScript DOM.</p>
    <button id="theme-toggle">Bật Chế độ Tối</button>
  </div>
  <script src="script.js"></script>
</body>
```

THAO TÁC VỚI HTML DOM

Ví dụ: tạo chế độ Light/Dark Mode

Mã CSS

```
/* Định nghĩa biến màu mặc định (Light Mode) */
:root {
  --bg-color: #ffffff;
  --text-color: #333333;
  --btn-bg: #222222;
  --btn-text: #ffffff;
}

/* Định nghĩa biến màu khi ở Dark Mode */
body.dark-mode {
  --bg-color: #1a1a1a;
  --text-color: #f0f0f0;
  --btn-bg: #f0f0f0;
  --btn-text: #1a1a1a;
}
```

```
body {
  background-color: var(--bg-color);
  color: var(--text-color);
  transition: 0.3s;
  font-family: Arial, sans-serif;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

button {
  padding: 10px 20px;
  cursor: pointer;
  background-color: var(--btn-bg);
  color: var(--btn-text);
  border: none;
  border-radius: 5px;
}
```

THAO TÁC VỚI HTML DOM

Ví dụ: tạo chế độ Light/Dark Mode

Mã JavaScript

```
const btn = document.getElementById('theme-toggle');
const currentTheme = 'dark'; // Chế độ mặc định
if (currentTheme === 'dark') {
    document.body.classList.add('dark-mode');
    btn.innerText = "Bật Chế độ Sáng";
}
// Lắng nghe sự kiện Click
btn.addEventListener('click', () => {
    // Toggle class dark-mode trên thẻ body
    document.body.classList.toggle('dark-mode');
    let theme = 'light';
    if (document.body.classList.contains('dark-mode')) {
        theme = 'dark'; btn.innerText = "Bật Chế độ Sáng";
    } else {btn.innerText = "Bật Chế độ Tối"; }
    currentTheme = theme;
});
```

Kết quả trên trình duyệt

Chế độ Sáng / Tối

Đây là ứng dụng demo đơn giản dùng JavaScript DOM.

Bật Chế độ Sáng

Chế độ Sáng / Tối

Đây là ứng dụng demo đơn giản dùng JavaScript DOM.

Bật Chế độ Tối

AJAX = **A**synchronous **J**avaScript **A**nd **X**ML

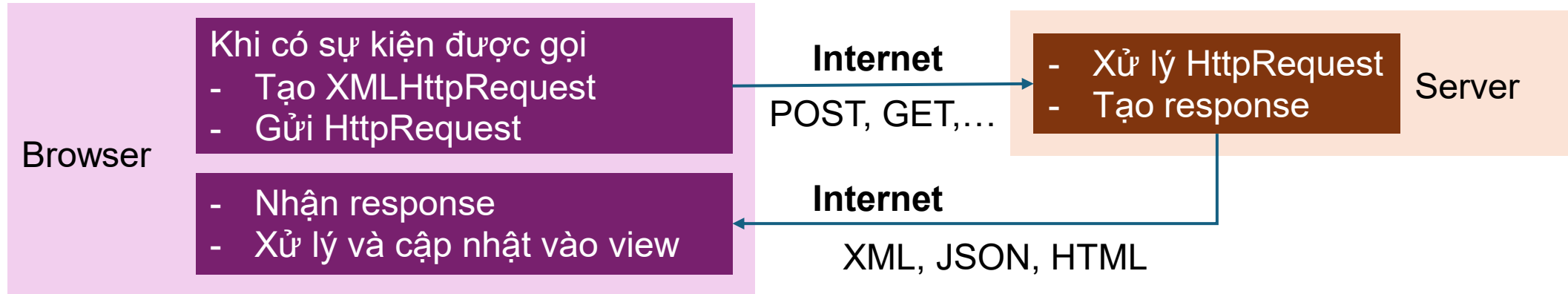
AJAX là một phương thức cho phép trao đổi dữ liệu với máy chủ mà không cần tải lại toàn bộ trang web, giúp cập nhật một hoặc nhiều phần của trang một cách linh hoạt và nhanh chóng.

Ajax được viết bằng Javascript chạy trên client, tức là mỗi browser sẽ chạy độc lập hoàn toàn không ảnh hưởng lẫn nhau.

AJAX hoạt động dựa trên việc gửi các yêu cầu HTTP từ phía client đến server ở chế độ nền, sau đó cập nhật từng phần của trang web mà không cần tải lại toàn bộ trang.

Các phương thức HTTP/HTTPs gồm: GET, POST, PUT, DELETE, PATCH

Đối tượng XMLHttpRequest (XHR) – Công nghệ cốt lõi của AJAX



Trong XHR, ta cần quan tâm đến các con số này để biết trạng thái XHR:

- 0 (Unsent): Vừa tạo đối tượng.
- 1 (Opened): Đã gọi hàm open().
- 2 (Headers_received): Đã gửi đi và Server đã nhận được.
- 3 (Loading): Server đang xử lý và chuẩn bị gửi dữ liệu về.
- 4 (Done): Hoàn tất.

Đối tượng XMLHttpRequest (XHR) – Công nghệ cốt lõi của AJAX

```
const xhr = new XMLHttpRequest(); // Bước 1: Khởi tạo đối tượng XHR

xhr.setRequestHeader("Content-Type", "application/json"); // Thiết lập các header nếu cần

// Bước 2: Cấu hình yêu cầu (Phương thức, URL, Bất đồng bộ?)
xhr.open('GET', 'https://api.example.com/data', true);

// Bước 3: Định nghĩa hàm xử lý khi thư hồi âm (Sự kiện onreadystatechange)
xhr.onreadystatechange = function () {
    // readyState 4: Yêu cầu đã hoàn thành - status 200: Server trả về thành công
    if (xhr.readyState === 4 && xhr.status === 200) {
        const data = JSON.parse(xhr.responseText);
        console.log("Dữ liệu nhận được:", data);
    } else if (xhr.readyState === 4) {
        console.error("Lỗi khi lấy dữ liệu:", xhr.statusText);
    }
};

xhr.send(); // Bước 4: Nhấn nút gửi thư đi
```

Fetch API – AJAX hiện đại.

Ngày nay, người ta không còn dùng XMLHttpRequest cũ kỹ nữa. Fetch API là chuẩn hiện đại, cho phép ta thực hiện các yêu cầu HTTP một cách đơn giản và mạnh mẽ hơn rất nhiều.

Fetch API sử dụng phương thức `fetch()`. Tham số đầu tiên là URL của tài nguyên ta muốn lấy.

```
fetch('https://api.example.com/data')
  .then(response => {
    // Chỗ này để xử lý phản hồi ban đầu
  })
  .then(data => {
    // Chỗ này để dùng dữ liệu cuối cùng
  })
  .catch(error => {
    // Chỗ này để bắt lỗi
  });
```

Fetch API – AJAX hiện đại.

Xử lý đối tượng Response (Phản hồi)

```
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(response => {
    if (!response.ok) { // Kiểm tra status từ 200-299
      throw new Error('Mạng có vấn đề!');
    }
    return response.json(); // Phải "giải mã" Body từ JSON sang JS Object
  })
  .then(data => {
    console.log("Dữ liệu nhận được:", data)
  })
  .catch(err => console.log(err));
```

File JSON là một định dạng dữ liệu được viết tắt từ JavaScript Object Notation. JSON có cấu trúc dựa trên các cặp khóa-giá trị, cho phép tổ chức dữ liệu một cách rõ ràng và có thể được sử dụng trong nhiều ngôn ngữ lập trình khác nhau.

Fetch API – AJAX hiện đại.

Gửi dữ liệu với tham số thứ hai là một Object cấu hình

```
const postData = {
  title: 'Học Fetch cơ bản',
  body: 'Fetch cơ bản cho mọi người',
  userId: 1
};

fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST', // Phương thức
  headers: {
    'Content-Type': 'application/json' // Báo cho Server biết mình gửi dữ liệu JSON
  },
  body: JSON.stringify(postData) // Chuyển Object thành chuỗi JSON
})
.then(res => res.json())
.then(data => console.log('Đã tạo bài viết:', data));
```

Fetch API – AJAX hiện đại.

Sử dụng async/await

```
async function getMyData() {  
  try {  
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');  
  
    if (!response.ok) throw new Error("Lỗi kết nối!");  
  
    const data = await response.json();  
    console.log("Dữ liệu nhận được:", data);  
  
  } catch (error) {  
    console.error("Gặp lỗi rồi:", error.message);  
  }  
}  
  
getMyData();
```

JQuery

NỘI DUNG CHÍNH

Giới thiệu về
jQuery

jQuery Selector
và Events

jQuery HTML

jQuery AJAX

GIỚI THIỆU CƠ BẢN VỀ JQUERY

jQuery là một thư viện JavaScript viết theo phương châm: "Write less, do more" (Viết ít hơn, làm nhiều hơn).

- Nhiệm vụ: Giúp thao tác với DOM, xử lý sự kiện, hiệu ứng và AJAX trở nên cực kỳ ngắn gọn.
- Đặc điểm: Tương thích tốt với mọi trình duyệt cũ (nối âm ảnh của JS thuần ngày xưa).

Cài đặt: (Phiên bản mới nhất 4.0.0 tại thời điểm 2026)

```
<script src="https://code.jquery.com/jquery-4.0.0.js"></script>
```

Hoặc có thể download tại trang chính để dùng offline: <https://jquery.com/>

GIỚI THIỆU CƠ BẢN VỀ JQUERY

Cú pháp mặc định (Document Ready):

Để đảm bảo code jQuery chỉ chạy khi toàn bộ HTML đã tải xong:

```
<script>
    $(document).ready(function(){
        console.log("jQuery đã sẵn sàng!");
    });
</script>
```

Hoặc viết tắt

```
<script>
    $(function(){
        console.log("jQuery đã sẵn sàng!");
    });
</script>
```

jQuery đã sẵn sàng!

Selector - Sức mạnh của dấu \$

jQuery dùng cú pháp Selector giống hệt CSS, cực kỳ dễ nhớ:

- `$("p")`: Chọn tất cả thẻ `<p>`.
- `$(".test")`: Chọn tất cả thẻ có class là test.
- `$("#name")`: Chọn thẻ có id là name.
- `$("ul li:first")`: Chọn thẻ li đầu tiên trong ul.

Một số ví dụ về JQuery Selectors

<code>\$("*")</code>	Chọn tất cả phần tử
<code>\$(this)</code>	Chọn phần tử HTML hiện tại
<code>\$("p.intro")</code>	Chọn tất cả thẻ <code><p></code> với <code>class="intro"</code>
<code>\$("p:first")</code>	Chọn thẻ <code><p></code> đầu tiên
<code>\$("ul li:first-child")</code>	Chọn thẻ <code></code> đầu tiên của toàn bộ thẻ <code></code>
<code>\$("[href]")</code>	Chọn tất cả các phần tử có một thuộc tính <code>href</code>
<code>\$("a[target='_blank']")</code>	Chọn tất cả thẻ <code><a></code> có thuộc tính <code>target</code> là <code>"_blank"</code>
<code>\$("a[target!='_blank']")</code>	Chọn tất cả thẻ <code><a></code> có thuộc tính <code>target</code> khác <code>"_blank"</code>
<code>\$(":button")</code>	Chọn tất cả <code><button></code> và <code><input></code> có <code>type="button"</code>
<code>\$("tr:even")</code>	Chọn tất cả <code><tr></code> chẵn
<code>\$("tr:odd")</code>	Chọn tất cả <code><tr></code> lẻ

Sự kiện (Events)

jQuery được thiết kế riêng để phản hồi các sự kiện mà người dùng hành động trong một trang HTML. Thay vì addEventListener dài dòng như JavaScript thuần, jQuery có cú pháp rất gọn:

```
$("#button").click(function(){  
    $(this).hide(); // Bấm vào nút nào, nút đó biến mất  
});
```

Sự kiện (Events)

Sự kiện chuột (Mouse Events):

- `.click()`: Xảy ra khi nhấn chuột trái vào phần tử.
- `.dblclick()`: Xảy ra khi nhấn đúp chuột (2 lần liên tiếp).
- `.mouseenter()`: Xảy ra khi con trỏ chuột bắt đầu đi vào vùng của phần tử.
- `.mouseleave()`: Xảy ra khi con trỏ chuột rời khỏi phần tử.
- `.mousedown()`: Xảy ra khi nhấn nút chuột xuống (chưa thả ra).
- `.mouseup()`: Xảy ra khi thả nút chuột ra.
- `.hover(in, out)`: Một hàm tiện ích kết hợp giữa `mouseenter` và `mouseleave`.

Sự kiện (Events)

Sự kiện Bàn phím (Keyboard Events):

- `.keypress()`: Xảy ra khi một phím được nhấn xuống và giữ (thường dùng cho các phím ký tự).
- `.keydown()`: Xảy ra ngay khi phím vừa được nhấn xuống (bắt được tất cả các phím bao gồm Ctrl, Alt, Shift).
- `.keyup()`: Xảy ra khi người dùng thả phím ra.

Sự kiện (Events)

Sự kiện Form (Form Events):

- `.submit()`: Xảy ra khi người dùng nhấn nút gửi Form.
- `.change()`: Xảy ra khi giá trị của ô Input, Checkbox hoặc Select bị thay đổi.
- `.focus()`: Xảy ra khi người dùng nhấn vào ô Input (con trỏ chuột nằm trong ô).
- `.blur()`: Xảy ra khi người dùng nhấn ra ngoài ô Input (mất tiêu điểm).

Sự kiện (Events)

Sự kiện trình duyệt:

- `$(document).ready()`: Xảy ra khi cây DOM đã được tải xong (an toàn để chạy code jQuery).
- `$(window).resize()`: Xảy ra khi người dùng thay đổi kích thước cửa sổ trình duyệt.
- `$(window).scroll()`: Xảy ra khi người dùng cuộn trang web (thường dùng làm hiệu ứng "Back to top" hoặc thanh menu dính).

Sự kiện (Events)

Quản lý nhiều sự kiện bằng .on():

```
$("#p").on({  
  mouseenter: function(){  
    $(this).css("background-color", "lightgray");  
  },  
  mouseleave: function(){  
    $(this).css("background-color", "lightblue");  
  },  
  click: function(){  
    $(this).css("background-color", "yellow");  
  }  
});
```

Sự kiện (Events)

Ngăn chặn hành động mặc định:

Đôi khi em muốn click vào thẻ `<a>` nhưng không muốn nó chuyển trang, hoặc nhấn nút Submit nhưng không muốn trang tải lại:

- `event.preventDefault()`: Ngăn hành động mặc định của trình duyệt.
- `event.stopPropagation()`: Ngăn sự kiện lan tỏa (bubbling) lên các phần tử cha.

Lấy và Thay đổi nội dung (Get/Set Content)

jQuery cung cấp 3 phương thức để tương tác với nội dung bên trong các thẻ:

- `.text()`: Trả về hoặc thiết lập nội dung văn bản thuần túy (loại bỏ các thẻ HTML).
- `.html()`: Trả về hoặc thiết lập nội dung bao gồm cả các thẻ HTML bên trong.
- `.val()`: Trả về hoặc thiết lập giá trị của các trường trong Form (như `<input>`, `<select>`, `<textarea>`).

Thêm phần tử mới (Add Elements)

Đây là phần thú vị nhất, giúp ta tạo ra các nội dung động (như thêm dòng mới vào giỏ hàng). Có 4 phương thức chính:

- `.append()`: Chèn nội dung vào cuối bên trong phần tử được chọn.
- `.prepend()`: Chèn nội dung vào đầu bên trong phần tử được chọn.
- `.after()`: Chèn nội dung vào sau (bên ngoài) phần tử được chọn.
- `.before()`: Chèn nội dung vào trước (bên ngoài) phần tử được chọn.

Xóa phần tử (Remove Elements)

Khi người dùng nhấn nút "Xóa" một món hàng, ta sẽ dùng:

- `.remove()`: Xóa bỏ hoàn toàn phần tử được chọn (và tất cả các phần tử con của nó).
- `.empty()`: Chỉ xóa các phần tử con bên trong (giữ lại cái "vỏ" của phần tử được chọn).

Thao tác với Class và CSS

Thay vì can thiệp trực tiếp vào style (inline style), các lập trình viên chuyên nghiệp thường tạo sẵn các class trong CSS và dùng jQuery để "bật/tắt" chúng.

- `.addClass()`: Thêm một hoặc nhiều class.
- `.removeClass()`: Xóa class.
- `.toggleClass()`: Chế độ "công tắc" - nếu có class rồi thì xóa, chưa có thì thêm. (Cực kỳ hữu ích cho nút Dark Mode hoặc Menu di động).
- `.css()`: Thiết lập hoặc trả về thuộc tính style trực tiếp.

Thao tác với kích thước (Dimensions)

jQuery giúp việc tính toán kích thước của các phần tử trở nên cực kỳ đơn giản, không cần lo lắng về margin hay padding phức tạp.

- width() & height()
- innerWidth() & innerHeight() (Bao gồm Padding)
- outerWidth() & outerHeight() (Bao gồm cả Padding và Border)

Thay vì dùng JavaScript thuần (fetch hoặc XHR), jQuery AJAX mang lại các lợi ích:

- Cú pháp ngắn gọn: Viết ít hơn, làm được nhiều hơn.
- Tương thích trình duyệt: jQuery tự xử lý các vấn đề khác biệt giữa các trình duyệt.
- Dễ quản lý: Các hàm callback như success, error, complete giúp luồng xử lý cực kỳ rõ ràng.

Các phương thức cơ bản:

- \$.load() – Phương thức đơn giản nhất để lấy dữ liệu
- \$.get() - Lấy dữ liệu từ Server
- \$.post() - Gửi dữ liệu lên Server
- \$.ajax() - Đây là phương thức mạnh mẽ nhất, cho phép cấu hình mọi chi tiết của yêu cầu.

Server thường trả về dữ liệu dạng JSON (JavaScript Object Notation). jQuery sẽ tự động chuyển chuỗi JSON này thành Object để ta dễ dàng truy cập.

Phương thức .load():

Đây là cách nhanh nhất để lấy nội dung HTML từ một trang khác và đổ thẳng vào một phần tử trên trang hiện tại. Cú pháp:

```
$(selector).load(URL, [data], [callback]);
```

Trong đó:

- URL (Bắt buộc): Đường dẫn đến tệp hoặc server cần lấy dữ liệu. Có thể load một phần tử cụ thể bằng cách thêm selector vào URL:

```
$("#result").load("test.html #content");
```

- data (Tùy chọn): Cặp key/value gửi kèm cùng yêu cầu.
- callback (Tùy chọn): Hàm được thực thi sau khi hoàn tất tải dữ liệu.

Phương thức .load():

Hàm callback bao gồm các tham số.responseText (dữ liệu), textStatus (trạng thái), và XMLHttpRequest (đối tượng).

Ví dụ: Tải nội dung file thông-bao.html vào thẻ div có id là content.

```
$("#content").load("thong-bao.html", function(response, status, xhr) {  
    if (status == "success") {  
        console.log("Tải dữ liệu thành công!");  
    }  
});
```

Phương thức .get():

Dùng để gửi yêu cầu lấy dữ liệu từ Server. Thông tin gửi đi sẽ hiển thị trên URL (query string). Chẳng hạn sử dụng .get() khi Lấy danh sách sản phẩm, tìm kiếm thông tin theo tên. Cú pháp sử dụng:

```
$.get(url, [data], [callback], [dataType])
```

Trong đó:

- URL (Bắt buộc): Địa chỉ máy chủ nơi gửi yêu cầu.
- Data (Tùy chọn): Dữ liệu gửi kèm theo dạng chuỗi hoặc đối tượng.
- Callback (Tùy chọn): Hàm xử lý được gọi khi yêu cầu thành công, thường nhận về dữ liệu phản hồi (data), trạng thái (status).
- DataType (Tùy chọn): Loại dữ liệu mong đợi từ server (xml, json, script, text, html).

Phương thức .get():

Ví dụ: Lấy thông tin sinh viên theo ID

```
$.get("get_student.php", { id: 101 }, function(data) {  
    // Giả sử data trả về là: "Sinh viên: Nguyễn Văn An"  
    $("#result").text(data);  
});
```

Phương thức .post():

Dùng để gửi dữ liệu lên Server để xử lý. Dữ liệu được gửi "ngầm", không hiện lên URL, bảo mật hơn GET. Thường sử dụng khi gửi form đăng ký, đăng nhập, thêm bình luận. Cú pháp sử dụng:

```
$.post(url, [data], [callback], [dataType])
```

Trong đó:

- URL (Bắt buộc): Địa chỉ máy chủ nơi gửi yêu cầu.
- Data (Tùy chọn): Dữ liệu gửi kèm theo dạng chuỗi hoặc đối tượng.
- Callback (Tùy chọn): Hàm thực thi khi yêu cầu thành công (thường là `function(data, status, xhr)`).
- DataType (Tùy chọn): Định dạng dữ liệu mong muốn nhận về (như `xml`, `json`, `script`, `text`, `html`).

Phương thức .post():

Ví dụ: Gửi một bình luận mới.

```
$.post("save_comment.php",  
    { name: "An", content: "Bài giảng rất hay!" },  
    function(response) {  
        alert("Server phản hồi: " + response);  
    }  
);
```

Phương thức .ajax():

Phương thức \$.ajax() trong jQuery là công cụ cốt lõi, mạnh mẽ nhất để thực hiện các yêu cầu HTTP không đồng bộ. Nó linh hoạt hơn so với các phương thức cấp cao như \$.get() hoặc \$.post() nhờ khả năng cấu hình chi tiết, giúp quản lý các yêu cầu phức tạp một cách hiệu quả.

Phương thức .ajax():

Cú pháp cơ bản:

```
$.ajax({  
  url: 'url_dich_vu', // Đường dẫn server  
  type: 'POST',       // Hoặc 'GET', 'PUT', 'DELETE'  
  data: { id: 1 },    // Dữ liệu gửi đi  
  dataType: 'json',   // Kiểu dữ liệu mong đợi  
  beforeSend: function() { ... }, // Hàm chạy trước khi yêu cầu được gửi.  
  success: function(response) { ... }, // Xử lý khi thành công  
  error: function(xhr, status, error) { ... } // Xử lý khi lỗi  
});
```

Ví dụ: Hệ thống đăng nhập chuyên nghiệp.

```
$.ajax({
    url: "login_process.php", // Đường dẫn xử lý
    type: "POST",             // Phương thức
    data: { user: $("#user").val(), pass: $("#pass").val() },
    dataType: "json",         // Mong muốn nhận về dữ liệu dạng JSON
    beforeSend: function() { $("#spinner").show(); }, // Hiện icon xoay xoay khi đang đợi
    success: function(res) {
        if (res.status === "success") {
            window.location.href = "dashboard.php";
        } else {
            alert("Sai mật khẩu rồi!");
        }
    },
    error: function(xhr, status, error) { console.error("Lỗi kết nối: " + error); },
    complete: function() {
        $("#spinner").hide(); // Luôn ẩn icon xoay xoay dù thành công hay thất bại
    }
});
```

Giới thiệu một số thư viện JavaScript

Đây là nhóm giúp ta tạo ra các ứng dụng Web phức tạp (Single Page Application - SPA).

- React (by Meta): Phổ biến nhất hiện nay. Nó sử dụng cơ chế Virtual DOM và kiến trúc Component, giúp ta tái sử dụng code cực kỳ hiệu quả.
- Vue.js: Được mệnh danh là "dễ học nhất" trong bộ ba quyền lực. Tài liệu của Vue rất thân thiện và phù hợp cho người mới bắt đầu.
- Angular (by Google): Một framework "toàn năng" (all-in-one). Nó rất chặt chẽ, thường được dùng trong các dự án ngân hàng hoặc hệ thống doanh nghiệp lớn.

Những thư viện này giúp ta xử lý logic "ngầm" một cách mượt mà.

- Axios: Thay thế cho fetch API. Nó giúp việc gọi AJAX trở nên chuyên nghiệp hơn với khả năng tự động chuyển đổi dữ liệu JSON và quản lý lỗi (interceptors) cực tốt.
- Lodash: Một "túi thần kỳ" chứa hàng trăm hàm xử lý Mảng (Array), Đối tượng (Object), Chuỗi (String)... giúp code của ta ngắn gọn và tối ưu hơn rất nhiều.
- TanStack Query (React Query): Thư viện "vàng" để quản lý trạng thái dữ liệu từ Server, tự động hóa việc caching và đồng bộ dữ liệu.
- Toastify JS: Một thư viện JavaScript nhẹ giúp bạn tạo toast notification (thông báo nhỏ dạng popup) một cách nhanh chóng mà không cần phụ thuộc vào framework như React hay Vue.

Nếu bạn muốn trang web của mình trông thật "xịn xò" và chuyên nghiệp.

- Chart.js / D3.js: Dùng để vẽ các biểu đồ, đồ thị từ dữ liệu. D3.js cực mạnh nhưng khó học, trong khi Chart.js dễ dùng cho các nhu cầu cơ bản.
- GSAP (GreenSock): Thư viện làm animation (chuyển động) mạnh mẽ nhất thế giới web hiện nay. Hầu hết các trang web đoạt giải thưởng lớn đều dùng GSAP.
- Three.js: Nếu bạn muốn mang thế giới 3D vào trình duyệt (như các game web hoặc showroom 3D), đây là lựa chọn số 1.
- Swiper: Mạnh mẽ và linh hoạt dùng để tạo slider hoặc carousel trên giao diện web. Nó được thiết kế để dễ dàng sử dụng, nhanh chóng và không yêu cầu cài đặt phức tạp. Swiper hỗ trợ nhiều tính năng như điều hướng, phân trang, hiệu ứng chuyển động, và tương thích tốt với các thiết bị di động.

NHÓM CSS-IN-JS & GIAO DIỆN (UI COMPONENTS)

Thay vì viết CSS thủ công, ta có thể dùng các thư viện này để "lắp ghép" giao diện một cách nhanh chóng.

- Tailwind CSS: Mặc dù là thư viện CSS, nhưng nó gắn liền với hệ sinh thái JS hiện đại. Nó giúp cho việc thiết kế giao diện cực nhanh bằng các class tiện ích.
- MUI (Material UI) / Ant Design: Cung cấp sẵn các nút bấm, ô nhập liệu, menu... đã được thiết kế đẹp mắt theo chuẩn Google hoặc Alibaba.
- TinyMCE: Là thư viện rich text editor viết bằng JavaScript, cho phép người dùng nhập và định dạng văn bản giống như trong Word.

Để đảm bảo code của bạn không có lỗi trước khi giao cho khách hàng.

- Jest: Thư viện kiểm thử (Testing) phổ biến nhất, giúp ta tự động chạy các kịch bản để kiểm tra logic của mình.
- Cypress: Dùng để kiểm thử toàn diện quy trình (End-to-End), ví dụ: Tự động giả lập người dùng vào web, gõ mật khẩu và nhấn đăng nhập xem có thành công không.

TÀI LIỆU THAM KHẢO

- [1] JavaScript tutorial: <https://www.w3schools.com/js/default.asp>
- [2] JQuery tutorial: <https://www.w3schools.com/jquery/default.asp>

THỰC HÀNH HỖN HỢP

Tiếp tục thực hiện UI/UX cho các trang trong dự án web của nhóm.

Các nội dung gợi ý:

- Form validation
- Quản lý giỏ hàng
- Tạo slide đầu trang
- Tạo menu ẩn/hiện cho mobile
- Slide ảnh sản phẩm
- Tính năng phân trang
- ...

THANKS

